
What Counts as a Mistake? Annotating Recitation Events in Quran Memorization Transcripts

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Checking Quran recitation from an ASR transcript requires distinguishing unre-
2 solved mistakes from repetitions, repairs, opening formulas and accepted spelling
3 differences. We report a completed human annotation of 100 production record-
4 ing cases: 348 scored units and 162 localized events across ten combined la-
5 bels. An executable evaluator scores labels and word positions together. A plain
6 diff reaches label-aware F1 0.525 and localization F1 0.826; adapted production
7 cleaner/alignment components reach 0.518 and 0.786, with exact-span F1 0.505
8 for both. Correcting the adapter’s word coordinates recovers all five annotated
9 repetition events, showing why annotation interfaces must be checked before in-
10 terpreting baseline failures. In a preliminary pilot, seven single 20-minute runs
11 across three coding agents and seven models span label-aware F1 0.143 to 0.892:
12 six land far above every baseline, and one collapses below the naive diff from
13 a missing normalization step. Across the six, 970 of 972 gold-event instances
14 draw an overlapping prediction, so what remains is not detection but convention:
15 span extent, and the labels whose boundary is stipulated by adjudication rather
16 than visible in the text. Seven of 162 events defeat all six runs, five of them one
17 orthographic rule. No run annotated any training data, so the pilot measures the
18 algorithm half of the task only.

19 1 The distinction the task turns on

20 Quran memorization is normally practised with a listener who corrects the reciter. An application
21 that stands in for that listener transcribes with commercial ASR and compares the transcript against
22 the reference. Edit distance locates differences but does not determine their interpretation. In our
23 production data 74% of reviewed ayah units carry no event at all, and among those that do, a
24 substantial share are things a human listener would never mention: the reciter repeated a phrase after
25 a breath, said a word wrongly and immediately fixed it, or used a written form differing from the
26 Uthmani orthography without differing in what was recited. Flagging these as unresolved mistakes
27 can give the learner misleading feedback; we do not measure the relative user cost here.

28 We therefore annotate transcript-against-reference differences as typed, localized *events*, and we
29 judge the text rather than the speaker: a label records what differs between transcript and reference,
30 not whether the reciter or the ASR caused it. Unwritten vowel and tajweed errors lie outside a
31 transcript-only rubric and outside this work. Three things follow: an annotated corpus of production
32 recitations, a label scheme whose distinguishing move is that a repetition, a repair and an accepted
33 spelling variant are each their own category rather than degrees of error, and a first measurement
34 of what a coding agent does when handed the unlabelled data and asked to solve annotation and
35 algorithm together.

2 Related work

Machine support for Quran recitation has mostly worked from audio: verse delimitation with speech recognition [Tabbal et al., 2006], tajweed-rule classification from the acoustic signal [Alagrami and Eljazzar, 2020], and pretrained encoders for Quranic ASR [Hossain et al., 2026], reviewed by Balula et al. [2021]. The datasets follow the same shape: Tarteel crowd-sources and labels recitations [Khan et al., 2021], a later corpus labels non-Arabic speakers’ recitations in six categories with inter-rater agreement [Salameh et al., 2024], and Quran-MD aligns verse and word audio with text [Salman et al., 2025]. All mark a recording or a phone correct or incorrect; we label the transcript, typing and locating every difference. Pronunciation assessment scores a learner attempt against a reference [Witt and Young, 2000], and disfluency annotation records a repetition or a repair as its own structure rather than as an error [Shriberg, 1994]; our rubric applies the second idea to an ASR transcript against a fixed reference. The coding agents we score are normally measured on software and ML engineering benchmarks [Jimenez et al., 2024, Chan et al., 2025].

3 Taxonomy and annotation

Combined labels. An earlier pilot used two parallel fields, an event type and a benign/mistake verdict, plus an uncertain escape hatch. Reviewing real recordings made both untenable: the verdict was never independent of the type, and uncertain absorbed exactly the cases that needed a decision. The rubric now uses one combined label per event, `substitution_mistake`, `omission_mistake`, `insertion_mistake`, `substitution_corrected`, `omission_corrected`, `repetition_benign`, `letters_benign`, `spelling_benign`, `basmala_benign` and `isti3adha_benign`, with `clean` for a reviewed unit carrying no event. A unit reviewed and found clean is not the same as a unit not yet reviewed, and the data distinguishes them.

Four decisions that took adjudication. *Omissions* include missing beginnings, interiors and endings, with no exemption for the final ayah of a recording: stopping early still leaves reference material absent from the transcript, and a reviewer cannot infer intent from recording position. *Repeats* span the original phrase together with its extra occurrence against a single reference copy, aligning their localization with corrected events; a wrong-then-correct attempt is a corrected event, while a correct-then-incorrect restatement is a `substitution_mistake` spanning both attempts, with the order recorded in the note. *Spelling* differences removed by an agreed context-independent normalization receive no event, and only a residual form needing contextual acceptance receives `spelling_benign`; the accepted contexts are enumerated rather than generalized, so the category cannot quietly become blanket final-letter deletion. *Localization* uses verbatim words and half-open spans into the original whitespace token arrays, with an empty hypothesis span anchoring an omission and an empty reference span anchoring an insertion.

How the annotation was produced. One reviewer approved every record, working from the unchanged ASR transcript beside the exact reference for each ayah, and approval is per whole recording rather than per event: a recording with any unresolved question stays out of the set instead of entering it with a hedged label. That is why the rubric has no `uncertain` category and does not need one. Uncertainty is expressed by withholding approval. All 314 ayah units and 34 opening units in the set carry that approval, and every one of the 115 within-ayah events carries a written note recording which difference was the target and why the label was chosen. Those notes are what made the failure analysis in Section 6 possible: for each event a run got wrong we can read the reasoning the run was implicitly being asked to reproduce. They stay private with the answers.

Scale and composition. Table 1 gives the label counts, alongside how many teaching examples each label has and how the six working pilot runs of Section 6 handled it. All 100 selected recording cases are human-approved: 314 ayah units, 233 of them clean, and 34 opening units, over 274 ayahs in 38 surahs. Ten learner identities contribute, one of them 46 recordings. Selection was purposive, so these counts describe the annotated set; they do not estimate error prevalence or imply generalization to unseen reciters. Three further approved recordings remain outside the scored set; their transcript-only copies join the development candidates, with annotations archived privately. We verified identifiers and transcripts against both source exports and excluded punctuation-normalized duplicates and redundant clean subpassages. Gold answers and selected membership remain in an owner-held private bundle; this draft does not release a gold sample.

Label	gold	teach	correct	mislab.	missed	dominant failure
substitution_mistake	62	7	94%	0%	6%	span too narrow
isti3adha_benign	29	0	100%	0%	0%	—
omission_mistake	26	5	88%	0%	12%	read as substitution
basmala_benign	18	0	88%	0%	12%	compound opening
spelling_benign	12	1	24%	61%	15%	read as substitution
repetition_benign	5	1	63%	0%	37%	read as insertion
substitution_corrected	4	1	38%	0%	62%	read as insertion
letters_benign	4	1	54%	46%	0%	read as substitution
insertion_mistake	1	1	17%	0%	83%	read as substitution
omission_corrected	1	1	0%	67%	33%	read as corrected substitution

Table 1: Gold events per label, teaching examples supplied per label, and the outcome over the six working pilot runs (162 gold events $\times$ 6 = 972 instances; 84.0% correct, 6.1% localized but mislabelled, 10.0% unmatched). *Correct* requires both the label and a span pair above tolerance. Reviewed clean units carry no event and are not rows here.

90 4 An executable scorecard

91 Systems receive the reviewed ayah split, the ayah ids and the exact references, so detection and
92 segmentation are supplied rather than scored; that task is a companion submission. Per unit a
93 system returns events of the form {label, hyp_span, ref_span}, with half-open spans into the
94 transcript and reference token arrays.

95 **Opening formulas are scored.** Each record’s isti’adhah and basmala are presented as their own unit,
96 with the formula as the transcript and an empty reference; a compound opening yields two events
97 over one token array. Deciding that a basmala preceding the ayah is benign rather than an insertion is
98 the judgment under test; excluding it would hide the failure mode that matters most.

99 **Matching.** Predictions and gold events are paired one-to-one within a unit by maximum total
100 similarity, so a single broad prediction is credited with at most one gold event and the rest count as
101 misses. A pair is eligible only when *both* spans overlap, $\min(\text{sim}_{hyp}, \text{sim}_{ref}) \geq 0.50$; averaging
102 the two, as an earlier version did, lets an exact reference span carry a match while the hypothesis
103 span points elsewhere. Events are validated for known labels, integer endpoints, token bounds and
104 label-specific empty spans. Invalid predictions count as false positives. Empty spans match only
105 empty anchors, exactly or within one token; they never match selected words. Pairing ignores labels
106 so localization can be reported separately. Units with more than seven events on either side use
107 deterministic greedy matching.

108 **Scores.** The primary measure is label-aware event F1: a paired prediction is a true positive only
109 when its label also matches, and a mislabeled pair counts once as a false positive and once as a false
110 negative. We report micro F1 as the headline, exact-span F1 requiring identical endpoints, macro F1
111 across labels present in gold, and localization precision/recall/F1 with labels dropped. We also report
112 an application-facing cost in raw event counts, (r missed + false flags) per 100 units, at $r = 1$ and
113 $r = 2$, because no exchange rate between a missed mistake and an unnecessary flag is established.
114 Neither rate ranks systems and the ordering is unchanged from 1:1 to 5:1; the rate matters only in that
115 predicting nothing wins below roughly 0.77:1. Predicting nothing gives F1 0 and the gold annotation
116 gives 1. Evaluator v2.1 passes synthetic coordinate, label, anchor and adapter regression tests and
117 private gold oracle checks. Sweeping the tolerance from 0.05 to 1.00 moves baseline micro F1 by
118 about four points and leaves exact-span F1 unchanged, since diff-based baselines emit exact spans;
119 0.50 is chosen to be explainable rather than tuned. One-token anchor slack changes no baseline score
120 across 0 to 10 and is retained because the index of an omission between two tokens is genuinely
121 ambiguous. A system predicting looser spans will depend on the tolerance more than these baselines.

122 5 What existing systems do

123 The plain diff locates 126 of 162 gold events (recall 0.778), with localization F1 0.826. Its label-aware
124 recall is 0.494 and F1 is 0.525. These quantities differ: localization F1 is not the percentage of gold
125 events found. It reaches per-label F1 0.94 for omissions and 0.85 for substitutions, and zero for the
126 other eight labels. It emits 46 insertion predictions, 34 of them on opening-text units, whereas gold

System	micro F1	exact F1	macro F1	loc F1	review cost
Predict nothing	0.000	0.000	0.000	0.000	51.2
Plain diff	0.525	0.505	0.178	0.826	23.3
Adapted production components	0.518	0.505	0.284	0.786	26.7
Codex, GPT-6-Astra (pilot)	0.892	0.862	0.701	0.934	8.1
Claude Code, Opus 5 (pilot)	0.880	0.849	0.660	0.931	9.2
Codex, GPT-5.6-Sol (pilot)	0.875	0.863	0.631	0.938	9.5
Claude Code, Sonnet 5 (pilot)	0.856	0.825	0.609	0.912	9.8
Antigravity, Gemini 3.6 Flash (pilot)	0.797	0.773	0.387	0.878	14.1
Antigravity, Gemini 3.1 Pro (pilot)	0.795	0.677	0.502	0.873	10.9
Antigravity, gpt-oss-120B (pilot)	0.143	0.121	0.017	0.187	169.3

Table 2: Evaluator v2.1: 348 units, 162 gold events. No run crashes; the gpt-oss-120B run produces 22 invalid events, discussed below. The seven pilot rows are one 20-minute development run per agent/model pair, not the preregistered comparison of Section 8.

has one insertion event. Compound openings contain two gold formulas but a diff can return one insertion spanning both.

The second baseline adapts production cleaner and alignment components to the new event schema; the deployed application does not natively expose it. The adapter retains original token identities through deletion, uses alignment indices and reference offsets, and maps cleaner notes to reference phrases and both attempts. Its heuristic interprets a stumble as a repair only when the following equally long surviving span matches the reference; unlocalizable removed fragments remain extra text. Production’s partial-start scoring policy is retained. These choices are part of the baseline, not additional gold decisions.

This adapter predicts five repetitions and recovers all five (F1 1.0); six corrected-substitution predictions recover one of four gold events (F1 0.20). It has nonzero F1 on four labels and zero on six, and never predicts either opening-formula label, letters, contextual spelling or omission corrections. The earlier adapter lost valid repetition credit through incorrect coordinates. The remaining label gaps motivate explicit event modeling, but they do not establish that cleaning causes the errors, or measure every feature of the deployed application. Each baseline retains its own comparison normalization, an algorithm choice rather than a change to the gold rubric.

6 A preliminary agent pilot

Each of three coding agents (Codex, Antigravity, Claude Code) was given the unlabelled 127-recording training release, the annotation guide and 20 constructed teaching examples with answers, a 20-minute development budget, and no access to gold or the internet; each froze one `solution.py` that we ran, unmodified, over the private gold set. Codex ran under GPT-5.6-Sol and GPT-6-Astra; Antigravity under Gemini 3.1 Pro, Gemini 3.6 Flash and gpt-oss-120B; Claude Code under Opus 5 and Sonnet 5, once each. Seven runs, seven models, one run per model: this is not the repeated-run comparison of Section 8, no model, budget or seed was frozen in advance, and nothing here ranks agents or models in general.

Six of the seven reach micro F1 0.79–0.89, far above every baseline, with zero invalid predictions across all 348 units. The spread between them is smaller than the gap from either baseline, and with one run per model it should not be read as an ordering.

Where the six runs fail, and why

Table 1 splits every gold event by outcome. Detection is not the residual problem: across 972 gold-event instances, 970 drew an overlapping prediction, and only two, both `spelling_benign`, went entirely unnoticed. What the runs lose is convention. Of the 97 unmatched instances, 95 had a prediction overlapping the gold span that fell below the matching tolerance, most often because the rubric puts both attempts of a repeat or a repair in one event while the run emitted the extra text alone; that single convention accounts for the `substitution_corrected` and `repetition_benign` columns, where the runs read the first attempt as an insertion. The 59 mislabelled instances concentrate in `spelling_benign` and `letters_benign`, both read as `substitution_mistake`.

Only seven of the 162 gold events defeated all six runs. Five are the same orthographic case, a final alif maqsura not sustained in wasl before a following definite article, which the rubric enumerates as accepted in named contexts and which all six runs called `substitution_mistake`. One is a restart supplying a missing preposition, annotated `omission_corrected`, which four runs read as `substitution_corrected`; the alternative is defensible on the transcript alone. The last is a two-attempt phrase where the gold span covers both attempts and every run marked only the second. Teaching coverage does not explain the pattern. The two opening-formula labels have no teaching example at all and are handled at 100% and 88%, while `spelling_benign`, which has one, is handled at 24%. What separates them is whether the boundary is structural, as an opening formula in its own unit is, or stipulated by adjudication, as the accepted spelling contexts are. Stipulated boundaries are the part a rubric has to carry, and one worked example does not carry them.

How the six runs agree with each other

Volume is not the problem either. The six runs emit 145 to 172 events against 162 in gold, and on the 233 reviewed clean units they raise three spurious mistake flags between them: one each for Sonnet 5, Gemini 3.6 Flash and Gemini 3.1 Pro, none for the other three. False positives run from 16 to 31 per run and are dominated by `substitution_mistake` in every case. Whatever these runs get wrong, they are not flooding a learner with invented mistakes, which is the failure mode the application can least afford.

Run	events predicted	false positives	clean units flagged
Codex, GPT-6-Astra	172	23	0
Claude Code, Opus 5	156	16	0
Codex, GPT-5.6-Sol	158	18	0
Claude Code, Sonnet 5	158	21	1
Antigravity, Gemini 3.6 Flash	159	31	1
Antigravity, Gemini 3.1 Pro	145	23	1

Table 3: Error structure of the six working runs against 162 gold events on 348 units, 233 of them clean. Every run is calibrated on volume to within eleven percent of the gold count.

Treating the six runs as independent readers of the same rubric gives a second view of the annotation. On 114 of the 162 gold events (70%) all six agree with the gold label. On 43 (27%) they split. On five (3%) all six agree with each other and disagree with gold: four are the `spelling_benign` wasl case read as `substitution_mistake`, and one is the two-attempt phrase every run localized too narrowly. Unanimous disagreement is therefore not noise. It landed on one rubric rule and one span convention, and the rubric’s own notes describe that rule as an approved extension of an earlier allowance rather than something a reader could derive. Consensus across independently developed solutions is a cheap signal for where a rubric is doing stipulative work, though it cannot decide whether the stipulation is right.

The seventh run is an informative failure. `gpt-oss-120B`’s solution never normalizes Arabic orthography before diffing, so ordinary hamza and alif-form differences register as `substitution_mistake` on nearly every token (545 of 567 predicted events carry that one label); it also treats every opening unit as clean and produces 22 invalid events. At micro F1 0.143 it is worse than either baseline: a diff-shaped answer under the same instructions, guide and examples as the six successful runs can still fail on the comparison step that Section 3’s rubric assumes is solved.

7 Discussion

What the label scheme buys. The scheme’s distinguishing move is that a repetition, a repair and an accepted spelling variant are categories in their own right rather than milder mistakes. That move is not cosmetic on this data. Of the 162 located events, 68 are benign and 5 are corrected, so 73 of them, 45%, would reach a learner as unresolved mistakes under any scheme that only has mistake categories, on top of the 233 units that carry no event at all. The distinction also survives contact with systems: the two baselines score 0.826 and 0.786 on localization but only 0.525 and 0.518 label-aware, and the gap is entirely naming, not finding. A checker that reports where the transcript

differs from the reference is a solved problem. A checker that reports what a listener would have said is not.

What the loop did, and what it did not. Handed the unlabelled pool, the guide and 20 worked examples, six of seven runs produced code that reaches 0.79 to 0.89 within 20 minutes, against 0.525 for a plain diff and 0.518 for adapted production components. The result is close enough to the ceiling that the interesting question is no longer whether the loop works on the algorithm but where its remaining errors come from, and Section 6 answers that: convention rather than detection. The runs are also conservative in the way the application needs, raising three spurious mistake flags across 1,398 clean-unit decisions.

The annotation half is a different story, and the honest answer is that this pilot did not test it. Every run went straight to rules; not one labelled any of the 888 training units. A 20-minute budget makes that rational, so nothing here says an agent will not annotate under a budget that rewards it. It does say that offering annotation as an available strategy is not enough to observe it, and that a design meaning to study the annotate-then-develop schema has to require delivery of the annotations and give the loop time to make producing them worthwhile.

Where the two halves meet. The failure analysis suggests the two problems are not independent. Every label the runs handle well has a boundary visible in the input: an opening formula occupies its own unit, missing reference material is missing. Every label they handle badly has a boundary fixed by adjudication: which spelling differences are accepted in which contexts, how far a span reaches when a reciter attempts a phrase twice. An agent asked to annotate would face exactly the boundaries its algorithm gets wrong, and the five events where all six runs agree against gold point at the same places. That makes agent consensus useful as a rubric audit before it is useful as a labelling shortcut: it says nothing about which reading is correct, but it locates the clauses where the rubric is stipulating rather than describing, and those are the clauses a human has to own.

8 Limitations and what comes next

The annotated set is purposive and small, one learner contributes 46 of the 100 recordings, and the label distribution is heavily skewed, so macro F1 here is a coverage check, not a stable estimate. A single reviewer approved every record, so we report no inter-annotator agreement and cannot separate the rubric from that reviewer’s reading of it. The written notes make each decision auditable and the failure analysis shows where independent readers would diverge, but neither substitutes for a second annotator on a sample, which is the first thing we would add. The five events where all six runs agree against gold are the natural place to start. More consequentially, all 100 selected transcripts already appear as inputs in a public release of the same recordings, and twelve in an earlier machine-labeled pilot. The adjudications and the selection membership are private, but the inputs are not, and runtime isolation cannot establish absence of prior exposure; an unseen-input claim requires a separate corpus. The prepared development release contains 127 cases and 888 units, including 44 of 63 newer recordings. It permits shared clean ayahs and different error variants, but excludes complete gold copies and event-bearing gold chunks. The one local ASR repair appears non-Quranic and is excluded. Remaining new recordings are not a frozen hidden test.

We plan to compare agents under the same unlabelled training data, rubric, examples, tools and budget. Agents may annotate, train models or write rules; annotation is an available strategy, not the experimental contrast. In the pilot, none of the seven runs took it: every workspace we recovered contains rule-based code and a self-test against the supplied examples, and not one labelled any part of the 888-unit training pool. Under a 20-minute budget that is a rational choice, and it means this pilot measures the algorithm half of the task only. Whether an agent will annotate at all, and whether its labels survive review, needs a budget that makes annotating worth the time and a delivery requirement that makes it observable. The pilot above is a first, informal instance of this design, run to check that the harness and isolation work end to end before the preregistered protocol is fixed. Isolation differed by runtime. Codex and Claude Code each ran non-interactively in a fresh git-free directory outside this repository, under their sandbox’s default network restriction, with an isolated credentials-only home directory carrying no prior session history, cache or goals, one fresh instance per model. Antigravity ran in a fresh window and directory per model; its network access was not independently instrumented, and its selected model is taken on the operator’s report rather than verified externally. All seven saw only the unlabelled release, the guide and the teaching examples,

never gold, and froze their solution before any gold scoring. A high score supports the combined process, not each training label: code may ignore or correct its labels. Generated annotations need independent review, but a fully human-labelled training pool is unnecessary. Gold answers, selected test bundles and review history stay outside development environments; shared clean inputs and prior public exposure are disclosed. The repeated-run comparison, with budgets, models and annotation delivery fixed in advance, remains future work.

References

- Ali M Alagrami and Maged M Eljazzar. Smartajweed automatic recognition of arabic quranic recitation rules. *arXiv preprint arXiv:2101.04200*, 2020.
- Nazik O’mar Balula, Mohsen Rashwan, and Shrief Abdou. Automatic speech recognition (asr) systems for learning arabic language and al-quran recitation: a review. *International Journal of Computer Science and Mobile Computing*, 10(7):91–100, 2021.
- Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, et al. Mle-bench: Evaluating machine learning agents on machine learning engineering. In *International Conference on Learning Representations*, volume 2025, pages 50466–50494, 2025.
- Nabil Mosharrar Hossain, Riasat Islam, and Unaizah Obaidellah. A comparative study of pretrained transformer models for quranic asr: Speech representations, label formats, and dataset composition. *arXiv preprint arXiv:2606.19747*, 2026.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, volume 2024, pages 54107–54157, 2024.
- Hamzah I Khan, Abubakar Abid, Mohamed Medhat Moussa, and Anas Abou-Allaban. The tarteel dataset: crowd-sourced and labeled quranic recitation. 2021.
- Raghad Salameh, Mohamad Al Mdfaa, Nursultan Askarbekuly, and Manuel Mazzara. Quranic audio dataset: Crowdsourced and labeled recitation from non-arabic speakers. *Procedia Computer Science*, 246:2684–2693, 2024.
- Muhammad Umar Salman, Mohammad Areeb Qazi, and Mohammed Talha Alam. Quran-MD: A fine-grained multimodal dataset of the quran. In *5th Muslims in ML Workshop co-located with NeurIPS 2025*, 2025.
- Elizabeth Ellen Shriberg. Preliminaries to a theory of speech disfluencies. *Doctoral dissertation, University of California at Berkeley*, 1994.
- Hassan Tabbal, W El Falou, and B Monla. Analysis and implementation of a" quranic" verses delimitation system in audio files using speech recognition techniques. In *2006 2nd international conference on information & communication technologies*, volume 2, pages 2979–2984. IEEE, 2006.
- Silke M Witt and Steve J Young. Phone-level pronunciation scoring and assessment for interactive language learning. *Speech communication*, 30(2-3):95–108, 2000.